

嵌入式期末试卷II

2.任务有什么特征，任务与进程的区别，任务与函数的区别。概念题

这又是一道非常经典的操作系统底层概念题。在嵌入式实时操作系统（RTOS）的语境下，要把这三个概念区分清楚，关键要抓住“执行流”、“内存空间”和“上下文”这三个核心点。

以下是为你梳理的答题要点：

1. 任务（Task）有什么特征？

在 RTOS 中，任务是可以独立运行的最小程序单元。它通常具有以下几个核心特征：

- **动态性：** 任务是一个运行时的概念，它是代码在数据集合上的一次执行过程，有创建、运行、阻塞、挂起和删除等生命周期，而不是一段静态的代码。
- **独立性：** 每个任务都有自己独立的“私有财产”，即**独立的任务控制块（TCB）和独立的任务堆栈（Stack）**。从任务自身的视角来看，它会认为自己独占了整个 CPU。
- **并发性（或宏观并行性）：** 在操作系统的调度下，多个任务可以在极短的时间片内交替占用 CPU，使得宏观上看起来就像是多个任务在同时运行。
- **异步性：** 任务按照各自独立的、不可预知的速度向前推进。如果没有操作系统的同步机制（如信号量、互斥锁），任务之间是互不干涉的。

2. 任务（Task）与进程（Process）的区别

在通用操作系统（如 Linux）中，我们常说“进程”和“线程”；但在诸如 uC/OS-II 等单片机 RTOS 中，通常只提“任务”。在 RTOS 的语境下，“任务”其实更接近于 Linux 中的“线程”。

- **内存空间隔离性（最本质区别）：**
 - **进程：** 拥有独立的虚拟地址空间（通常依赖硬件 MMU 支持）。进程与进程之间的内存是物理隔离的，一个进程指针越界崩溃，通常不会影响其他进程。
 - **任务（RTOS中）：** 系统中所有的任务共享同一个单一的、平坦的物理内存空间（通常没有 MMU）。如果一个任务发生了野指针踩内存，极有可能直接把其他任务的数据或操作系统的内核数据破坏掉，导致系统死机。
- **切换开销：**

- **进程**：切换时不仅要保存 CPU 寄存器，还要切换页表目录、刷新 TLB（快表），开销非常大。
- **任务**：切换极其轻量级，仅仅是把 CPU 寄存器压入当前任务的私有堆栈，并从下一个任务的私有堆栈中弹出寄存器即可。

3. 任务（Task）与函数（Function）的区别

这是很多初学操作系统的人最容易混淆的地方，因为在 C 语言代码里，任务往往也是写成一个函数的样子（比如 `void TaskA(void *p_arg)`）。

- **调用与执行方式：**

- **函数**：是**串行、同步**调用的。函数 A 调用函数 B 时，必须通过 `BL` 或 `CALL` 指令跳转，并且 A 必须等待 B 执行完毕并 `Return` 之后，A 才能继续往下走。
- **任务**：是**并发、异步**运行的。任务由操作系统的“调度器”来决定何时执行。一个任务不能像调用函数那样去直接“调用”另一个任务，只能通过系统 API 唤醒它。

- **代码结构与生命周期：**

- **函数**：有明确的起点和终点，执行完特定的逻辑后，**必须返回（Return）**。
- **任务**：绝大多数情况下是一个**无限死循环（比如 `while(1)`**），永远不会返回。如果一个任务真的走到了大括号的尽头，必须调用系统 API（如 `OSTaskDel`）来把自身销毁掉，否则系统会跑飞。

- **栈空间的使用：**

- **函数**：共享调用它的那个执行流（进程、线程或中断）的堆栈。
- **任务**：必须拥有自己在创建时被分配的**绝对独立的私有堆栈**。任务里的局部变量全都保存在自己的堆栈中，与其他任务互不干扰。

- **状态机制：**

- **函数**：只有“正在执行”和“未执行”的物理状态。
- **任务**：受到操作系统的严格管理，具有明确的状态机机制（如就绪态、运行态、阻塞/延时态、挂起态）。

3. CPSR寄存器的N、Z、C、V分别的作用是什么，请根据图示汇编代码说说是如何依次提高代码运行效率的。

这是一个非常基础且极其重要的底层架构考点。CPSR（Current Program Status Register，当前程序状态寄存器）的高 4 位（条件标志位）是处理器做出逻辑判断的“眼睛”。

以下是这四个标志位的详细作用，以及结合底层特性的效率优化分析。

1. CPSR 条件标志位的作用

- **N (Negative - 负数标志)**

- **作用：**记录 ALU（算术逻辑单元）最近一次操作的结果是否为负数。
- **判断逻辑：**如果运算结果的最高位（Bit 31）为 1，则 N 置 1；如果为 0（即结果为正数或 0），则 N 置 0。

- **Z (Zero - 零标志)**

- **作用：**记录最近一次操作的结果是否绝对为 0。
- **判断逻辑：**运算结果的所有位全为 0 时，Z 置 1；否则 Z 置 0。这是判断两个数是否相等最常用的标志（例如 `CMP` 比较两个相等的数，相减为 0，Z 就会被置 1）。

- **C (Carry - 进位/借位/移位标志)**

- **作用：**记录无符号数运算时的进位或借位情况，或者移位操作的溢出位。
- **判断逻辑：**
 - 在**加法**（包含 `CMN`）中，如果产生了进位（超出了 32 位能表示的范围），C 置 1。
 - 在**减法**（包含 `CMN`）中，如果**没有**产生借位（即被减数 $\geq$ 减数），C 置 1；如果产生了借位，C 置 0。
 - 在**移位**（Shift）指令中，C 存放最后一次被移出寄存器的那个位的值。

- **V (Overflow - 溢出标志)**

- **作用：**记录**有符号数**运算时是否发生了溢出。
- **判断逻辑：**如果两个同号的数相加，结果变成了异号（例如正数加正数变成了负数）；或者两个异号的数相减，结果与被减数异号，说明溢出了，V 置 1。

2. 代码运行效率优化分析（基于常见 ARM 优化逻辑）

由于你没有提供或上传具体的汇编代码图片，我无法对特定的代码行进行推演。不过，在 ARM 体系结构中，利用 CPSR 标志位来“依次提高代码运行效率”的经典考核模型通常分为以下三个递进层次。你可以对照一下你的试卷或讲义，看看是否符合这个逻辑：

第一层：初级写法（传统的“比较 + 分支跳转”）

在这个阶段，代码通常是这样写的：

代码段

```
CMP R0, R1 ; 比较 R0 和 R1, 更新 CPSR 的 N, Z, C, V 标志
BEQ Label_1 ; 如果 Z=1 (相等), 则跳转执行特定代码
BNE Label_2 ; 如果 Z=0 (不等), 则跳转执行另一段代码
```

- **效率瓶颈：** 频繁使用跳转指令（B 系列）。在带流水线（Pipeline）的处理器（如 ARM9 的五级流水线）中，一旦发生跳转，CPU 之前预取指和译码的指令全部作废，必须清空流水线（Pipeline Flush）并重新取指，这会带来巨大的时间损耗。

第二层：进阶优化（利用 ARM 特有的“条件执行”）

为了消除跳转带来的流水线破坏，ARM 允许绝大多数指令带有条件码后缀。

代码段

```
CMP R0, R1 ; 更新 CPSR 标志位
MOVEQ R2, R3 ; 如果 Z=1 (相等), 直接执行此指令
ADDNE R2, R2, 1 ; 如果 Z=0 (不等), 直接执行此指令
```

- **效率提升：** 去掉了跳转指令（Branch）。CPU 顺序读取指令，如果不满足 CPSR 的条件标志，CPU 会直接将该指令视为空操作（NOP）滑过。**这彻底避免了流水线清空，极大地提升了执行效率。**

第三层：终极优化（巧用带 'S' 后缀的指令合并操作）

很多时候，为了循环控制或者判断，我们需要执行一条算术指令，然后再单独跟一条 **CMP** 指令来更新标志位。

- **优化前：** 代码段

```
SUB R0, R0, #1 ; 执行减法
CMP R0, #0 ; 专门为了判断是否减到 0 而更新 CPSR
BNE Loop
```

- **优化后（结合 'S' 后缀）：**

代码段

```
SUBS R0, R0, #1 ; 'S' 后缀表示：在执行减法的同时，根据运算结果直接更新 CPSR!
BNE Loop
...
```

* **效率提升：** 通过在普通指令后加上 `S`（如 `SUBS`，`ADDS`），让算术/逻辑指令直接接管更新 CPSR 的任务，**直接节省了一条 `CMP` 指令**。这既减小了代码体积（节省了 ROM 空间），又减少了时钟周期的消耗。

你能把题目具体提到的那张汇编代码图片发给我看看吗？这样我能为你进行最精确的步骤图解。

4.ERTOS采用了什么措施来保证它的实时性与确定性？

这道题和第一题问的“嵌入式实时系统”其实是同一个概念（**ERTOS** 即 Embedded Real-Time Operating System 的缩写）。虽然核心考点一致，但针对“**采用了什么措施**”这种问法，阅卷老师通常更希望看到从**系统底层架构和算法设计**的具体手段来作答。

你可以按照以下几个核心“措施”来组织答案，条理会非常清晰：

1. 采用基于优先级的抢占式 $O(1)$ 调度算法

- **措施：** ERTOS 放弃了通用操作系统追求公平的“时间片轮转”，而是实行严格的“优先级至上”策略。同时，内核在设计就绪表时（如 uC/OS-II 的优先级位图），采用查表法或硬件前导零指令。
- **保证效果：** 确保了系统无论有多少个任务处于就绪态，寻找最高优先级任务的算法执行时间都是 $O(1)$ （恒定常数）。只要高优先级任务一就绪，就能在绝对可预测的、极短的时间内抢占 CPU。

2. 采用定长内存池分配代替传统 malloc

- **措施：** 传统的 `malloc` 在碎片化的堆中寻找可用内存耗时不可知，且极易产生内存碎片。ERTOS 采取的措施是预先划分固定大小的**内存池（Memory Pool/Block）**，或者强制要求静态分配。并且 ERTOS 通常不使用带缺页异常的虚拟内存（MMU）。
- **保证效果：** 彻底杜绝了缺页中断带来的极高时间开销。分配和释放一块固定大小的内存块变成了简单的链表指针操作，执行时间是完全确定的，保证了系统的确定性。

3. 采用“优先级继承 / 优先级天花板”解决优先级反转

- **措施：** 在多任务争夺互斥资源（Mutex）时，为了防止高优先级任务被低优先级任务无限期阻塞（优先级反转），内核引入了优先级继承（Priority Inheritance）或天花板协议。临时将持有锁的低优先级任务的优先级提升到与高优先级任务相同。
- **保证效果：** 限制了高优先级任务被阻塞的最坏情况等待时间（WCET 具备上限），从而保证了系统在复杂的资源同步环境下的强实时性。

4. 严格控制内核的“关中断”时间

- **措施：** 在操作系统内核中，有些临界区代码必须独占 CPU（即关中断执行）。ERTOS 的开发者在编写内核时，采取的措施是将这些临界区压缩到极短。同时，推崇将中断处理分为“上半部（ISR）”和“下半部（延迟任务）”，在 ISR 中只做清状态等最简操作。

- **保证效果：**极大地降低了系统的**中断延迟（Interrupt Latency）**。保证了外部硬件事件（如定时器溢出、传感器报警）发生时，CPU 能够以微秒级的确定时间立刻响应。

5. 采用差分时间链与高精度 Tick 节拍

- **措施：**系统时钟节拍（Tick）的中断处理是耗时大户。ERTOS 采取如差分时间链（Delta List）等优秀的数据结构来管理延时任务和超时控制。
- **保证效果：**每次 Tick 中断到来时，无需遍历所有挂起的任务，只需对链表头部的差值进行减一操作。极大地缩短了时钟中断的处理时间，使得系统的绝对时间管理变得高度精确且耗时确定。

答题建议：

由于你前面已经做过了 uC/OS-II 的位图法、差分时间链等题目，这道题完全可以把前面的知识点当成论据填进来。比如在回答调度算法时带一句“如采用位图法”，回答定时器时带一句“如采用差分时间链”，这样能让答案看起来非常扎实，拿高分轻而易举。

5. 给出μC/OSii中IRQ的中断处理代码，让你根据代码讲解有操作系统的时候的IRQ处理流程

这道题完美地呼应了你之前问过的第 4 题（那段 S3C2440 的汇编代码）。这也是嵌入式操作系统课程中最核心的考点：**引入操作系统后，中断处理不再是简单的“打断-执行-返回”，而是深度绑定了操作系统的任务调度机制。**

为了让你在考试中拿到满分，我先给出一套标准的 μC/OS-II (基于 ARM 架构) 的 IRQ 中断处理模板代码（为了便于理解，做了必要的注释和精简），然后分步为你讲解。

1. μC/OS-II 中标准的 IRQ 处理代码 (ARM 汇编伪代码)

代码段

```
OS_CPU_IRQ_ISR:
; ===== 【阶段一：保护现场】 =====
STMFD    SP!, {R1-R3}          ; 暂存R1-R3到IRQ栈
SUB      R2, LR, #4             ; 计算被打断指令的精确返回地址
MRS      R3, SPSR               ; 保存被打断时的状态寄存器(CPSR)

; 切换到 SVC（管理）模式。因为任务的堆栈都在SVC模式下
MSR      CPSR_c, #(SVC32_MODE | NOINT)
STMFD    SP!, {R2}              ; 将返回地址压入当前任务的堆栈
STMFD    SP!, {R4-R12, LR}      ; 将其他通用寄存器压入当前任务堆栈
; ... (省略部分寄存器搬运代码)...
STMFD    SP!, {R3}              ; 将状态寄存器压入堆栈

; ===== 【阶段二：通知OS并保存TCB】 =====
LDR      R0, =OSIntNesting      ; 获取操作系统的中断嵌套层数变量
```



```

LDRB    R1, [R0]
ADD     R1, R1, #1          ; OSIntNesting++
STRB    R1, [R0]

CMP     R1, #1              ; 判断是否是第一层中断?
BNE     OS_CPU_IRQ_ISR_1    ; 如果不是第一层（发生嵌套），跳过保存栈指针

LDR     R0, =OSTCBCur       ; 如果是第一层中断，获取当前任务控制块指针
LDR     R1, [R0]
STR     SP, [R1]            ; 【关键】将当前堆栈指针 (SP) 保存到该任务的TCB中

OS_CPU_IRQ_ISR_1:
; ===== 【阶段三：执行用户 ISR】 =====
; 清除硬件中断标志位（芯片相关）
BL      C_Interrupt_Handler ; 跳转到用户用C语言编写的具体中断服务函数

; ===== 【阶段四：中断退出与调度】 =====
BL      OSIntExit           ; 【核心】调用操作系统的中断退出函数

; ===== 【阶段五：恢复现场】 =====
; 注意：此时的 SP 可能已经变成了另一个更高优先级任务的堆栈指针！
LDMFD   SP!, {R4}
MSR     SPSR_cxsf, R4       ; 从栈中弹出并恢复状态寄存器
LDMFD   SP!, {R0-R12, LR, PC}^ ; 弹出所有通用寄存器，并强制返回被打断的任务

```

2. 有操作系统时的 IRQ 处理流程讲解（考试答题要点）

在有操作系统（如 $\mu\text{C}/\text{OS-II}$ ）的环境下，IRQ 的处理流程必须严格按照以下五个步骤进行，答题时请重点突出它与“裸机（无 OS）”中断处理的本质区别：

步骤一：保护现场与模式切换 (Context Saving)

- **裸机区别：**裸机通常直接把寄存器压入系统的 IRQ 堆栈即可。但在 RTOS 中，每个任务都有自己的“私有堆栈”。
- **流程：**CPU 进入 IRQ 模式后，必须立刻关闭中断，提取出返回地址（LR-4）和状态字（SPSR）。然后强制切换到 SVC（管理）模式，把所有 CPU 寄存器（上下文）全部压入当前正在运行的任务的私有堆栈中，以保证该任务将来能完美复活。

步骤二：通知 OS 进入中断与更新 TCB (Interrupt Enter)

- **流程：**必须将系统的中断嵌套计数器 OSIntNesting 加 1（等同于调用 OSIntEnter()）。
- **关键点：**系统会判断 OSIntNesting 是否等于 1。如果等于 1，说明刚才打断的是一个普通的任务，此时必须把当前的堆栈指针（SP）保存到当前任务的 TCB（OSTCBCur）中；如果大于 1，说明打断的是另一个中断，就不需要再保存任务堆栈指针了。

步骤三：执行实际的中断服务函数 (Execute ISR)

- **流程：** 读取硬件寄存器，判断中断源，清除硬件中断标志，然后跳转执行真正的业务逻辑（比如读取串口数据、释放一个信号量唤醒某个任务等）。

步骤四：中断退出与可能引发的任务调度 (Interrupt Exit - 核心区别)

- **裸机区别：** 裸机在执行完 ISR 后，会直接恢复寄存器并返回原来的代码。但 RTOS 绝对不能直接返回！
- **流程：** 必须调用 `OSIntExit()` 函数。该函数会将 `OSIntNesting` 减 1。
- **核心逻辑：** 当嵌套层数减到 0 时，说明所有中断都处理完了。此时 `OSIntExit` 会检查：在刚才的中断服务中，是否有更高优先级的任务被唤醒了（变成就绪态）？如果有，`OSIntExit` 会直接触发一次中断级的任务调度（改变 `OSTCBCur` 并把 `SP` 指向新任务的堆栈）。

步骤五：恢复现场 (Context Restore)

- **流程：** 从当前的堆栈指针（`SP`）所指的内存中，依次弹出状态寄存器和通用寄存器，最后恢复 PC 指针。
- **奇妙之处：** 如果刚才 `OSIntExit` 没有引发调度，那么弹出的就是原来被打断任务的寄存器，一切照旧；如果引发了调度，那么此时弹出的将是那个“最高优先级任务”在很久以前挂起时保存的寄存器。一条弹栈指令下去，CPU 就直接“穿越”到了新的任务中去运行了。

💡 **总结一句话：** 带有 OS 的 IRQ 处理流程，本质上是在硬件中断处理的缝隙中，强行塞入了“任务上下文的保护与恢复”以及“抢占式任务调度”的逻辑。

7. 给了一个点灯一个灭灯的Task（就是上课PPT的那两个Task），让你画 `OSTimeDly(10)` 函数的流程图

这也是 uC/OS-II 期中考试里非常高频的一道画图题。题目背景是经典的“双任务交替运行”实验（Task1 点灯，Task2 灭灯）。

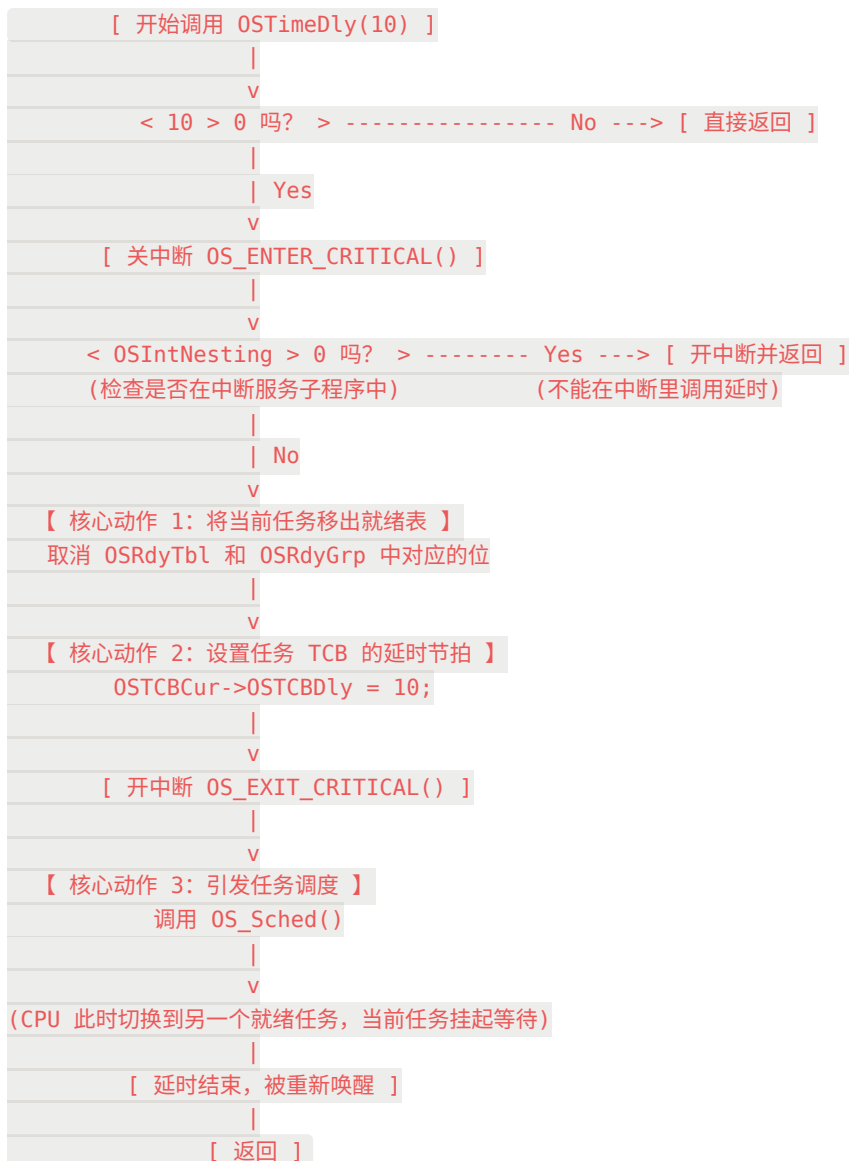
在 RTOS 中，任务不能使用简单的 `while/for` 循环来进行死延时（这会霸占 CPU），必须调用 `OSTimeDly()` 进行阻塞延时，主动交出 CPU 的使用权。

为了在考试中拿满分，你可以画出下面这个标准的逻辑流程图，并在旁边配上核心步骤的解释。

1. `OSTimeDly(10)` 内部执行流程图

在答题纸上，你可以用方框和菱形画出如下的结构：

Plaintext



2. 核心步骤图解与“踩分点”

如果在图的旁边加上以下几句核心解释，阅卷老师会认为你彻底懂了底层的调度逻辑：

- 参数检查与中断检查：** 系统首先会检查传入的 tick 数是否大于 0。接着必须检查 `OSIntNesting`，因为在中断服务函数（ISR）中是绝对不允许调用阻塞型延时函数的，否则会导致系统崩溃。
- 移出就绪表（关键）：** 当前点灯任务执行了延时，意味着它进入了“阻塞态”。操作系统会利用位操作（类似于你前面做过的位图法反向操作），将该任务在 `OSRdyTbl` 和 `OSRdyGrp` 中的相应标志位清零。这保证了在接下来的 10 个 tick 内，调度器都不会再多看它一眼。

3. **写入延时节拍数（关键）**：将传入的参数 `10` 赋值给当前任务控制块的延时变量 `OSTCBCur->OSTCBDly = 10`。在接下来的每一个系统硬件时钟节拍（SysTick）到来时，底层的时钟中断服务程序都会把这个值减 1。
4. **调用调度器（关键）**：准备工作做完后，打开中断，并立刻调用 `OS_Sched()` 进行任务调度。调度器会去查就绪表，找到当前就绪的最高优先级任务（比如此时那个灭灯的任务，或者空闲任务），并把 CPU 的执行权交给它。

💡 结合“点灯/灭灯”实验的补充说明：

当 Task1（点灯）调用 `OSTimeDly(10)` 后，它就被挂起了，CPU 立刻去执行 Task2（灭灯）。Task2 灭灯后也调用了 `OSTimeDly(10)`，CPU 就去执行底层的空闲任务（Idle Task）。随着底层硬件定时器不断发出“滴答”的中断，`OSTCBDly` 被不断减去。10 个 tick 后，减到了 0，底层的时钟服务函数会将 Task1 重新放回就绪表，Task1 再次抢占 CPU，开始新一轮的点灯。

8. 优先级位图法。写了代码

。解释你写的代码

给你一些插入任务的情景，让你写出每插入一个任务后优先级位图的变化

这道题是 uC/OS-II 操作系统中最核心、也最精妙的**优先级位图调度算法**的完整实战题。由于你在题目中没有附上具体的“插入任务情景”数值，我将为你提供**标准的核心代码、详尽的代码解释，并模拟一个典型的考场情景**为你推演整个过程。

为了让你在考前彻底弄懂这个机制，我还为你生成了一个**交互式的优先级位图模拟器**（见下方），你可以把你试卷上的具体数字输进去，直接看结果！

1. 核心代码（将任务插入就绪表的代码）

在 uC/OS-II 中，当一个任务准备好运行（进入就绪态）时，内核会执行以下两行极简且高效的代码将其插入位图：

C

```
// ptcb 是指向当前任务控制块（TCB）的指针
OSRdyGrp      |= ptcb->OSTCBBitY;
OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX;
```

2. 代码详细解释（踩分点）

这段代码的核心思想是**空间换时间**和**位操作**。为了实现 $O(1)$ 的调度时间，系统把所有的除法和求余运算都在任务创建时提前算好，存放在了 TCB（任务控制块）中。

- **OSRdyGrp（就绪组标志）**：一个 8 位的变量。每一位代表 `OSRdyTbl` 中的一行（即一组，包含 8 个优先级）是否有任务就绪。

- **OSRdyTbl (就绪表)**: 一个包含 8 个元素的数组, 每个元素也是 8 位。共 $8 \times 8 = 64$ 位, 完美对应 0~63 级优先级。
- **ptcb->OSTCBY**: 任务优先级的高 3 位。相当于 $Priority / 8$, 决定了任务属于哪个组 (即在 OSRdyTbl 的第几行)。
- **ptcb->OSTCBBity**: 组掩码。相当于 $1 \ll OSTCBY$ 。如果任务在第 1 组, 这个值就是 `0b00000010`。
- **ptcb->OSTCBBitX**: 位掩码。是由优先级的低 3 位 (相当于 $Priority \% 8$) 计算得出, 即 $1 \ll OSTCBX$ 。决定了任务在该组内的具体第几位。
- **|= (按位或运算)**:
 - 第一句: 将 OSRdyGrp 的相应位置 1, 宣告“这一组有任务就绪了”。
 - 第二句: 将 OSRdyTbl 对应行的相应位置 1, 宣告“该组内具体的这个优先级任务就绪了”。

3. 经典情景推演: 依次插入任务

假设系统初始状态为空 (`OSRdyGrp = 0b00000000`, `OSRdyTbl` 全部为 `0b00000000`)。

现在依次插入优先级为 12、21、10 的三个任务。

情景 1: 插入任务 12

1. 计算: $12 \div 8 = 1$ 余 4。所以 $Y=1$ (第 1 组), $X=4$ (第 4 位)。
2. 更新 OSRdyGrp: 第 1 位置 1。
 - `OSRdyGrp` 变为 `0b00000010`
3. 更新 OSRdyTbl: 第 1 组的第 4 位置 1。
 - `OSRdyTbl[1]` 变为 `0b00010000`

情景 2: 紧接着插入任务 21

1. 计算: $21 \div 8 = 2$ 余 5。所以 $Y=2$ (第 2 组), $X=5$ (第 5 位)。
2. 更新 OSRdyGrp: 第 2 位置 1。原来第 1 位已经是 1, 按位或之后保留。
 - `OSRdyGrp` 变为 `0b00000110`
3. 更新 OSRdyTbl: 第 2 组的第 5 位置 1。
 - `OSRdyTbl[2]` 变为 `0b00100000`

情景 3：紧接着插入任务 10

1. **计算：** $10 \div 8 = 1$ 余 2。所以 Y=1（第 1 组），X=2（第 2 位）。
2. **更新 OSRdyGrp：** 第 1 位置 1。此时 `OSRdyGrp` 的第 1 位原本就是 1，所以值不变。
 - `OSRdyGrp` 保持为 `0b00000110`
3. **更新 OSRdyTbl：** 第 1 组的第 2 位置 1。注意，此时 `OSRdyTbl[1]` 中原本的第 4 位（任务12）仍在。
 - `OSRdyTbl[1]` 变为 `0b00010100`

(你可以利用下面的交互组件，直接验证你试卷上的数字!)